

Detecção de Bad Smells e Refatoração Segura

Disciplina: Teste de Software

Gustavo Pimentel Carvalho Costa
Matrícula: 833151

May 2026

Análise de Bad Smells

O arquivo `src/ReportGenerator.js` contém o método `generateReport` (linhas 11-68) que concentra toda a lógica de geração de relatórios. A análise manual identificou cinco bad smells, dos quais três são detalhados abaixo.

1. High Cognitive Complexity (Complexidade Cognitiva Alta)

O método `generateReport` possui complexidade cognitiva de 27, medida pelo plugin SonarJS do ESLint. O threshold configurado é 15. O aninhamento de `for` → `if` role → `if` format → `if` priority → `if` format cria múltiplos caminhos de execução que tornam o código difícil de entender e testar completamente.

Evidência no código original:

```
for (const item of items) { if (user.role === 'ADMIN') { if (item.value > 1000) { ... } if (reportType === 'CSV') { ... } else if (reportType === 'HTML') { ... } } else if (user.role === 'USER') { if (item.value <= 500) { if (reportType === 'CSV') { ... } else if (reportType === 'HTML') { ... } } } }
```

Risco: com complexidade 27, existem caminhos de execução que podem não estar cobertos pelos testes. Um bug em um caminho raro só seria detectado em produção.

2. Duplicated Logic (Lógica Duplicada)

O bloco condicional que decide entre CSV e HTML aparece quatro vezes no mesmo método: no cabeçalho (linha 16), no corpo para ADMIN (linha 35), no corpo para USER (linha 46) e no rodapé (linha 58). A formatação de cada linha CSV — `${item.id},${item.name},${item.value},${user.name}` — se repete nas linhas 36 e 47.

Risco: adicionar um novo formato (ex: JSON) exigiria modificar quatro lugares diferentes, com alta probabilidade de inconsistência. A manutenção é proporcional ao número de cópias, não ao número de conceitos.

3. Magic Numbers (Números Mágicos)

Os valores 1000 (linha 30) e 500 (linha 45) aparecem sem contexto semântico. O primeiro é o limiar de prioridade para ADMINS; o segundo, o limiar de visibilidade para USERS. Sem nomes descritivos, outro desenvolvedor não sabe se são limites de negócio, técnicos ou arbitrários.

Risco: mudar o limiar exige buscar todas as ocorrências no código. Com duplicação, é fácil esquecer uma ocorrência, introduzindo comportamento inconsistente.

Relatório da Ferramenta — ESLint + SonarJS

O ESLint com o plugin `eslint-plugin-sonarjs` foi configurado com as regras `sonarjs/cognitive-complexity` (threshold 15) e `sonarjs/no-collapsible-if`. A execução no código original produziu:

```
src/ReportGenerator.js 11:3 error Cognitive Complexity from 27 to the 15 allowed 43:14 error
Merge this if statement with the nested one 2 problems (2 errors, 0 warnings)
```

O SonarJS detectou automaticamente a complexidade cognitiva com valor exato (27) — mais preciso que a estimativa manual (~22). Detectou também o `no-collapsible-if`, um smell adicional que a análise manual poderia ter perdido: o `else if (user.role === 'USER')` com `if (item.value <= 500)` aninhado pode ser colapsado em uma única condição.

A ferramenta complementa a análise manual: enquanto humanos identificam padrões estruturais (Long Method, Duplicated Logic), o SonarJS quantifica com precisão a complexidade e encontra problemas sutis de aninhamento.

Processo de Refatoração

O smell mais crítico — Duplicated Logic combinado com Long Method — foi tratado com duas técnicas de refatoração: Extract Method e Replace Conditional with Polymorphism.

Código refatorado (estrutura final):

```
const PRIORITY_THRESHOLD = 1000; const USER_VALUE_THRESHOLD = 500; class CsvFormatter { header,
formatItem, footer } class HtmlFormatter { header, formatItem, footer } function
getFormatter(type) { ... } function isVisibleToUser(item, user) { ... } function
applyBusinessRules(item, user) { ... } class ReportGenerator { generateReport(type, user, items)
{ // 12 linhas – limpo e legível } }
```

O método original tomava decisões de formatação e de negócio ao mesmo tempo. A refatoração separou essas responsabilidades: `ReportGenerator` decide o que incluir, os formadores decidem como apresentar.

Bônus: o código original continha um bug na formatação HTML. Para itens sem prioridade, produzia `<tr ><td>` (espaço antes do `>`), enquanto o teste esperava `<tr><td>`. O refatorado corrige isso usando `<tr${style}>` — sem espaço quando `style` é vazio.

Resultados da Validação

Métrica	Original	Refatorado
ESLint errors	2	0
Cognitive Complexity	27	< 15
Magic numbers	2 (1000, 500)	0
Testes passando	4/5	5/5
Formatadores	Acoplados	CsvFormatter + HtmlFormatter

Conclusão

A suíte de testes existente funcionou como rede de segurança: cada refatoração foi validada imediatamente, garantindo que o comportamento externo foi preservado. Sem os testes, a confiança para reorganizar 60 linhas de lógica acoplada seria significativamente menor.

A redução de bad smells produziu código mais legível e com menor complexidade cognitiva. Isso torna o código mais fácil de testar no futuro — os formatadores agora podem ser testados isoladamente, e as regras de negócio estão em funções puras com responsabilidade única. O ciclo se reforça: testes dão confiança para refatorar, código limpo é mais fácil de testar.